

MixBytes()

Lido ATokenWstETHSwap Security Audit Report

APRIL 21, 2026

Table of Contents

1. Introduction	2
1.1 Disclaimer	2
1.2 Executive Summary	2
1.3 Project Overview	3
1.4 Security Assessment Methodology	7
1.5 Risk Classification	9
1.6 Summary of Findings	10
2. Findings Report	11
2.1 Critical	11
2.2 High	11
2.3 Medium	11
M-1 ATokenWstETHSwap.setPremium() - Owner can frontrun any swap and extract up to 10% of the trade as "profit"	11
M-2 ATokenWstETHSwap.spell() - Arbitrary delegatecall lets the owner drain the vault allowance, live user approvals, and contract storage	12
2.4 Low	13
L-1 Small amountIn values round output to zero while the premium still pulls value out of the vault	13
L-2 _checkDebtCeiling() uses >= with off-by-one relative to maxSwapFromDebtCeiling()	14
L-3 Vault accrues loss when Aave's wstETH oracle diverges from stEthPerToken (depeg exposure)	15
L-4 availableWstETHLiquidity is an upper bound; maxSwapToWstETH result can shrink between quote and tx	16
3. About MixBytes	17

1. Introduction

1.1 Disclaimer

The audit makes no statements or warranties regarding the utility, safety, or security of the code, the suitability of the business model, investment advice, endorsement of the platform or its products, the regulatory regime for the business model, or any other claims about the fitness of the contracts for a particular purpose or their bug-free status.

1.2 Executive Summary

`ATokenWstETHSwap` is a one-way, fixed-rate aToken swap that lets Aave V3 WETH lenders exit illiquid `aEthWETH` positions – Aave's WETH market runs at ~100% utilization – into `aEthwstETH` that is pre-approved by a counterparty vault. The counterparty is expected to be a Mellow Core Vaults Subvault holding a leveraged `aEthwstETH` collateral / `variableDebtEthWETH` debt position on Aave V3. The swap values the output side at the `wstETH.stEthPerToken()` redemption rate, charges a configurable `premium` that is routed to a separate `profitReceiver`, and enforces a debt-ceiling invariant that keeps the vault a net WETH borrower after each swap.

The codebase was audited in 1 day by 3 auditors using a combination of manual review and automated tooling.

`ATokenWstETHSwap` is a single-route, fixed-rate aToken swap scoped to the `wstETH` leg only. Privileged addresses (`owner`, `vault`, `profitReceiver`) are `immutable` constructor arguments with zero-address validation, and the premium is bounded by a `MAX_PREMIUM = 1e5` (10%) constant enforced both at construction and in `setPremium`. The `spell` emergency primitive is retained with expanded NatSpec that documents the owner's god-mode privileges and warns that the owner must not be an EOA in production – no code-level mitigation is added. The pragma is pinned to `solidity 0.8.20` exactly.

The `MAX_PREMIUM` cap is a genuine hardening. The two Medium findings (`setPremium` frontrun and `spell`) are design-level residuals of the admin model and are not introduced by any coding defect.

The contract is live on Ethereum mainnet at `0x9D1F0f86E06f087E302E888aDBc9B07bfED0a25F` with constructor arguments (`owner = 0x199cC9fDD6c9a7d1924A32EA03097D7475FDbaDf`, `vault = 0x893aa69FBAA1ee81B536f0FbE3A3453e86290080`, `profitReceiver = 0xcCf2daba8Bb04a232a2fDA0D01010D4EF6C69B85`, `premium = 5000`) verified on-chain; `owner` is a 4-of-6 Gnosis Safe v1.4.1 and `profitReceiver` is a separate 4-of-7 Gnosis Safe v1.4.1 with no signer overlap.

Out of scope: Aave V3 protocol, Lido wstETH, Mellow Core Vaults contracts, and any off-chain components (UI, keepers, indexers).

1.3 Project Overview

Summary

Title	Description
Client	Lido
Category	Liquid Staking
Project	ATokenWstETHSwap
Type	Solidity
Platform	EVM
Timeline	20.04.2026 - 21.04.2026

Scope of Audit

File	Link
src/ATokenWstETHSwap.sol	src/ATokenWstETHSwap.sol

Versions Log

Date	Commit Hash	Note
20.04.2026	c63bbcaa60021a43c6d2fc30559c2159aa17dbae	Initial Commit
20.04.2026	00432c9a76049dfd9ab0804dfa892ff3ea4b89af	Commit with Updates

Mainnet Deployments

File	Address
ATokenWstETHSwap	0x9D1F0f86E06f087E302E888aDBc9B07bfED0a25F

[ATokenWstETHSwap](#) is deployed on Ethereum mainnet at [0x9D1F0f86E06f087E302E888aDBc9B07bfED0a25F](#), creation tx [0xb026af908e7a92605d3aa23d2fff882f632564dbcfa487b722fda23c6fa7ad6f](#). We verified the deployment: bytecode, source-verification metadata, constructor arguments and admin-address nature.

Bytecode and source verification. The Etherscan-verified source at [0x9D1F0f86E06f087E302E888aDBc9B07bfED0a25F](#) matches the audited [ATokenWstETHSwap.sol](#) at commit [00432c9a76049dfd9ab0804dfa892ff3ea4b89af](#). Etherscan reports compiler [v0.8.20+commit.a1b79de6](#), EVM version [shanghai](#), and optimizer disabled - consistent with the pinned [pragma solidity 0.8.20](#); in the audited source.

Deployer. The creation tx was sent directly from the EOA [0xF756993F7437EF868845C700cd46D5C677289039](#) (code size [0](#), nonce [2](#) at audit time) with the contract's creation bytecode in the transaction data. There is no intermediate deployer / factory contract: the `to` field of the creation tx is null and the contract address at [0x9D1F0f86E06f087E302E888aDBc9B07bfED0a25F](#) is the regular `CREATE` product of the EOA's address and nonce [1](#). This means the deployed bytecode is exactly what `solc` produced for the audited source - no factory-injected runtime preamble, no proxy indirection, and no initialization hook after construction.

Constructor arguments - verified on-chain. We queried each of the four public getters against the deployed contract and confirmed they match the stated deployment parameters exactly:

Field	On-chain (queried at deployed contract)
<code>owner()</code>	0x199cC9fDD6c9a7d1924A32EA03097D7475FDbaDf
<code>vault()</code>	0x893aa69FBAA1ee81B536f0FbE3A3453e86290080
<code>profitReceiver()</code>	0xcCf2daba8Bb04a232a2fDA0D01010D4EF6C69B85
<code>premium()</code>	5000
<code>paused()</code>	false

Constants (validating the compiled constants against [ATokenWstETHSwap.sol](#)):

Field	On-chain
<code>MAX_PREMIUM()</code>	100000
<code>PREMIUM_PRECISION()</code>	1000000
<code>aEthWETH()</code>	0x4d5F47FA6A74757f35C14fD3a6Ef8E3C9BC514E8
<code>aEthwstETH()</code>	0x0B925eD163218f6662a35e0f0371Ac234f9E9371

Field	On-chain
wstETH()	0x7f39C581F595B53c5cb19bD0b3f8dA6c935E2Ca0
variableDebtEthWETH()	0xeA51d7853EEFb32b6ee06b1C12E6dcCA88Be0fFE

The constructor's own sanity checks (`_owner/_vault/_profitReceiver` non-zero, `_premium <= MAX_PREMIUM`) are satisfied trivially for these values: all three addresses are non-zero and `5000 <= 100000`. The initial `premium` of 0.5% is well below the 10% hard cap, leaving substantial headroom before the `MAX_PREMIUM` bound.

`owner` is a Gnosis Safe multisig – not an EOA. We inspected the bytecode at `0x199cC9fDD6c9a7d1924A32EA03097D7475FDbaDf` and verified it is a Gnosis Safe proxy whose master copy is `0x41675C099F32341bf84BFc5382aF534df5C7461a` (Gnosis Safe v1.4.1 singleton).

On-chain queries:

- `getThreshold()` – 4
- `getOwners()` – 6 signers:
 - `0xa7615CD307F323172331865181DC8b80a2834324`
 - `0x1d895E5CF6E5288C9A56fACE942E016696Fb0C90`
 - `0x4604E3bFbaCcB317a6FfEde8Bf24105F476A7329`
 - `0x8888843c607F4Bbd6aD72128F478085256Bdd15D`
 - `0x59F8D74Fe49d5ebEAc069E3baf07eB4b614BD5A7`
 - `0x912e21CdA3D7012146da4Df33309d860a9eb0bEb`
- `nonce()` – 0 – the Safe has not executed any transaction yet as of the audit block.

This is a 4-of-6 Gnosis Safe v1.4.1. The choice materially improves the residual posture of finding M-2 (unscoped `spell` `delegatecall`) relative to an EOA owner: four independent signers would have to collude or be simultaneously compromised to exercise `spell`. The NatSpec warning "should never be an EOA in production" is honoured. Remaining in-contract gap: there is no on-chain timelock between a quorum decision and its effect, so a colluding quorum can still flip `premium` and revert within a single window – this is the residual of finding M-1 after the cap + multisig are stacked.

`profitReceiver` is a separate Gnosis Safe multisig.

`0xcCf2daba8Bb04a232a2fDA0D01010D4EF6C69B85` is also a Gnosis Safe v1.4.1 proxy pointing to the same `0x41675C099F32341bf84BFc5382aF534df5C7461a` singleton, with:

- `getThreshold()` – 4
- `getOwners()` – 7 signers:
 - `0xE6CD29BF19769E44534099d241abAa07aa8c8E9a`
 - `0x8aA493Bcd7f989caD67dd3CB48A4c2E6b40FeDB0`
 - `0x73B254db9cb71a129C4E73aF087D71Ac7d34a8d7`
 - `0xf2374BCb265505002055942D070459a4d2011012`
 - `0x75D95fF8D48E2Ca5c4235322A8AC8e52A76124cD`
 - `0x59d07dc34B135B17b87840a86BFF7302039E7EDf`
 - `0x8ecd93982FfbB2f937ADa6c6e50d1950974081C0`

This is a 4-of-7 Gnosis Safe v1.4.1, with no signer overlap with the `owner` Safe – the two roles are operationally separated. `profitReceiver` receives the premium portion of each swap in `aEthwstETH` and has no access to admin functions; separating it from `owner` is a reasonable custody split. Note that because the `profitReceiver` slot is `immutable`, it cannot be rotated without redeploying the swap contract – if the receiving Safe's signer set needs to be migrated, funds should first be swept out of that Safe before the redeploy, or a Safe signer rotation (which preserves the Safe's address) should be used instead.

`vault` is a `TransparentUpgradeableProxy` pointing into Mellow Core Vaults. `0x893aa69FBAA1ee81B536f0FbE3A3453e86290080` has 1107 bytes of proxy code; the EIP-1967 admin slot points to ProxyAdmin `0xd0a5294798b9d75062022535683afa06d13399af` and the EIP-1967 implementation slot points to the vanity address `0x0000000e535b4e063f8372933a55470e67910a66` (3420 bytes of logic code; the leading zero bytes are a `CREATE2`-mined vanity to reduce calldata gas when the address is referenced). Transparent proxy semantics mean external public reads from a non-admin caller cannot probe the implementation's getters through the proxy, which is why `name()` / `owner()` revert when called directly. The ProxyAdmin at `0xd0a5294798b9d75062022535683afa06d13399af` itself is owned by `0x81698f87C6482bF1ce9bFcFC0F103C4A0Adf0Af0`, which is a Gnosis Safe v1.3.0 with threshold 5-of-8. So the vault's upgrade authority is a separate Safe again (Safe v1.3.0 5-of-8), distinct from both `owner` and `profitReceiver`. An upgrade to a malicious implementation by that Safe would be able to move the vault's `aToken` balances and revoke the allowance granted to `ATokenWstETHSwap` – this is the standard Mellow Core Vaults trust model and is out of scope of the swap contract, but readers of this report should be aware that the vault-side balance held against the swap contract's allowance is subject to Mellow governance. CPIMP / re-initialisation – not applicable. `ATokenWstETHSwap` is a plain, non-upgradeable contract: all admin addresses are `immutable` and set only by the constructor, which runs exactly once in the creation transaction. There is no `initialize()`, no proxy, no reinitializer, and no post-construction init hook. The CPIMP attack class (front-running an `initialize()` on a freshly deployed proxy to seize admin roles) therefore does not apply. The only deploy-time risk surface is the choice of constructor arguments, and we have verified that each of the four supplied values matches the stated governance intent.

1.4 Security Assessment Methodology

Project Flow

Stage	Scope of Work
Interim audit	Project Architecture Review: • Review project documentation • Conduct a general code review • Perform reverse engineering to analyze the project's architecture based solely on the source code • Develop an independent perspective on the project's architecture • Identify any logical flaws in the design OBJECTIVE: UNDERSTAND THE OVERALL STRUCTURE OF THE PROJECT AND IDENTIFY POTENTIAL SECURITY RISKS.
	Code Review with a Hacker Mindset: • Each team member independently conducts a manual code review, focusing on identifying unique vulnerabilities. • Perform collaborative audits (pair auditing) of the most complex code sections, supervised by the Team Lead. • Develop Proof-of-Concepts (PoCs) and conduct fuzzing tests using tools like Foundry, Hardhat, and BOA to uncover intricate logical flaws. • Review test cases and in-code comments to identify potential weaknesses. OBJECTIVE: IDENTIFY AND ELIMINATE THE MAJORITY OF VULNERABILITIES, INCLUDING THOSE UNIQUE TO THE INDUSTRY.
	Code Review with a Nerd Mindset: • Conduct a manual code review using an internally maintained checklist, regularly updated with insights from past hacks, research, and client audits. • Utilize static analysis tools (e.g., Slither, Mythril) and vulnerability databases (e.g., Solodit) to uncover potential undetected attack vectors. OBJECTIVE: ENSURE COMPREHENSIVE COVERAGE OF ALL KNOWN ATTACK VECTORS DURING THE REVIEW PROCESS.

Stage	Scope of Work
	Consolidation of Auditors' Reports: • Cross-check findings among auditors • Discuss identified issues • Issue an interim audit report for client review OBJECTIVE: COMBINE INTERIM REPORTS FROM ALL AUDITORS INTO A SINGLE COMPREHENSIVE DOCUMENT.
Re-audit	Bug Fixing & Re-Audit: • The client addresses the identified issues and provides feedback • Auditors verify the fixes and update their statuses with supporting evidence • A re-audit report is generated and shared with the client OBJECTIVE: VALIDATE THE FIXES AND REASSESS THE CODE TO ENSURE ALL VULNERABILITIES ARE RESOLVED AND NO NEW VULNERABILITIES ARE ADDED.
Final audit	Final Code Verification & Public Audit Report: • Verify the final code version against recommendations and their statuses • Check deployed contracts for correct initialization parameters • Confirm that the deployed code matches the audited version • Issue a public audit report, published on our official GitHub repository • Announce the successful audit on our official X account OBJECTIVE: PERFORM A FINAL REVIEW AND ISSUE A PUBLIC REPORT DOCUMENTING THE AUDIT.

1.5 Risk Classification

Severity Level Matrix

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** – Theft from 0.5% OR partial/full blocking of funds (>0.5%) on the contract without the possibility of withdrawal OR loss of user funds (>1%) who interacted with the protocol.
- **Medium** – Contract lock that can only be fixed through a contract upgrade OR one-time theft of rewards or an amount up to 0.5% of the protocol's TVL OR funds lock with the possibility of withdrawal by an admin.
- **Low** – One-time contract lock that can be fixed by the administrator without a contract upgrade.

Likelihood

- **High** – The event has a 50-60% probability of occurring within a year and can be triggered by any actor (e.g., due to a likely market condition that the actor cannot influence).
- **Medium** – An unlikely event (10-20% probability of occurring) that can be triggered by a trusted actor.
- **Low** – A highly unlikely event that can only be triggered by the owner.

Action Required

- **Critical** – Must be fixed as soon as possible.
- **High** – Strongly advised to be fixed to minimize potential risks.
- **Medium** – Recommended to be fixed to enhance security and stability.
- **Low** – Recommended to be fixed to improve overall robustness and effectiveness.

Finding Status

- **Fixed** – The recommended fixes have been implemented in the project code and no longer impact its security.
- **Partially Fixed** – The recommended fixes have been partially implemented, reducing the impact of the finding, but it has not been fully resolved.
- **Acknowledged** – The recommended fixes have not yet been implemented, and the finding remains unresolved or does not require code changes.

1.6 Summary of Findings

Findings Count

Severity	Count
Critical	0
High	0
Medium	2
Low	4

Findings Statuses

ID	Finding	Severity	Status
M-1	<code>ATokenWstETHSwap.setPremium()</code> - Owner can frontrun any swap and extract up to 10% of the trade as "profit"	Medium	Acknowledged
M-2	<code>ATokenWstETHSwap.spell()</code> - Arbitrary <code>delegatecall</code> lets the owner drain the <code>vault</code> allowance, live user approvals, and contract storage	Medium	Acknowledged
L-1	Small <code>amountIn</code> values round output to zero while the premium still pulls value out of the vault	Low	Acknowledged
L-2	<code>_checkDebtCeiling()</code> uses <code>>=</code> with off-by-one relative to <code>maxSwapFromDebtCeiling()</code>	Low	Acknowledged
L-3	Vault accrues loss when Aave's wstETH oracle diverges from <code>stEthPerToken</code> (depeg exposure)	Low	Acknowledged
L-4	<code>availableWstETHLiquidity</code> is an upper bound; <code>maxSwapToWstETH</code> result can shrink between quote and tx	Low	Acknowledged

2. Findings Report

2.1 Critical

Not Found

2.2 High

Not Found

2.3 Medium

M-1	ATokenWstETHSwap.setPremium() - Owner can frontrun any swap and extract up to 10% of the trade as "profit"		
Severity	Medium	Status	Acknowledged

Description

`setPremium` is callable by `owner` at any block, with no timelock, no two-step pattern, and no snapshot binding `ATokenWstETHSwap.sol:203-207`. `swapToWstETH` reads `premium` from storage at execution time `ATokenWstETHSwap.sol:105-130`, so the owner can flip the premium to the 10% cap, land user swaps at that rate, and reset in the following block. `amountOutMin` cannot express "I'm OK with the premium I saw when I signed" because it conflates rate movement and premium movement.

Recommendation

Stack the 10% cap with an explicit `maxPremium` parameter on the swap entry points (so users bind the exact premium they saw when signing) and a two-step `propose` / `apply` timelock on `setPremium` so premium changes are publicly visible before they affect in-flight swaps.

Client's Commentary:

Risk accepted. Users are protected up to their slippage tolerance via `amountOutMin`. Premium flows to `profitReceiver`, which is a separate address from `owner`, so the owner role does not directly benefit from a premium-bump attack. Worst-case extraction per trade is bounded by `MAX_PREMIUM = 10%`. For additional safety, deployments may gate the owner behind a timelock.

M-2	ATokenWstETHSwap.spell() - Arbitrary <code>delegatecall</code> lets the owner drain the <code>vault</code> allowance, live user approvals, and contract storage		
Severity	Medium	Status	Acknowledged

Description

`spell(address target_, bytes memory data_)` executes a raw `delegatecall` to an arbitrary target with arbitrary calldata, guarded only by `onlyOwner ATokenWstETHSwap.sol:236-252`. Since this contract is the `spender` on the vault's `aEthwstETH` pre-approval and on every user's `aEthWETH` approval, a malicious `spell` payload can drain the entire vault allowance and every live user approval in one call. It can also overwrite storage (e.g. `premium`, `paused`) and bypass every in-contract invariant.

Residual exposure depends entirely on the owner address: with a timelocked multisig the risk is low; with an unlocked multisig it collapses to quorum-collusion; with an EOA the contract is equivalent to a custody relationship on any single-key compromise.

Recommendation

Replace the unconstrained `delegatecall` with a scoped rescue surface - a `rescueERC20(token, to, amount)` that only moves the contract's own balance, or at minimum a `call` with target and selector allowlists behind a timelocked multisig. If the primitive is kept, commit operationally to a timelock-gated owner and pre-announce every `spell` invocation.

Client's Commentary:

Risk accepted. `spell()` is a deliberate escape hatch for emergency corrective actions and is not expected to be used under normal operation. Access is restricted to `owner`, which should be set to a multisig on deployment. For additional security, a timelock may be used.

2.4 Low

L-1	Small <code>amountIn</code> values round output to zero while the premium still pulls value out of the vault		
Severity	Low	Status	Acknowledged

Description

`swapToWstETH` uses truncating integer division with `rate > 1e18` `ATokenWstETHSwap.sol:105-113`; the only input check is `amountIn != 0`. For `amountIn = 2` the user's output rounds to `0` while `profit` still rounds to `1` - the user loses 100% of the input. `amountOutMin = 0` passes trivially, silently flipping the "accept any rate" sentinel into "accept losing the whole input".

Impact is dust-scale in practice, but the contract makes no proportionality guarantee.

Recommendation

Revert with `ZeroAmount()` immediately after the `amountOut` assignment at `ATokenWstETHSwap.sol:111` when `amountOut == 0`, covering both the true-zero and rounded-to-zero cases.

Client's Commentary:

This is a rare edge case that requires the user to specifically submit dust-scale `amountIn` with `amountOutMin = 0`, bypassing the slippage protection already provided by the contract. Impact is wei-scale and isolated to the user. UI-layer checks can prevent such swaps from being created.

L-2	<code>_checkDebtCeiling()</code> uses <code>>=</code> with off-by-one relative to <code>maxSwapFromDebtCeiling()</code>		
Severity	Low	Status	Acknowledged

Description

The ceiling check reverts on `supply >= debt` `ATokenWstETHSwap.sol:257-261`; `maxSwapFromDebtCeiling` compensates by returning `debt - supply - 1` `ATokenWstETHSwap.sol:178-185`. The one-wei margin has no economic consequence, but integrators reproducing the formula may report a maximum that the contract rejects.

Recommendation

Either relax the check to `supply > debt` and drop the `-1` from `maxSwapFromDebtCeiling`, or document the one-wei margin in NatSpec on both sides so integrators reproduce the same bound.

Client's Commentary:

Both sides are internally consistent – `maxSwapFromDebtCeiling()` returns the exact maximum the contract will accept. It is up to the integrator to verify correctness when reproducing the formula off-chain.

L-3	Vault accrues loss when Aave's wstETH oracle diverges from <code>stEthPerToken</code> (depeg exposure)		
Severity	Low	Status	Acknowledged

Description

The swap values output at `wstETH.stEthPerToken()` while Aave values the same token via its own oracle. Today the two match within ~0.001% because Aave uses a ratio-based wstETH/ETH oracle derived from `stEthPerToken`. If Aave later switches to a market-price oracle (as governance has done and reverted before) and stETH depegs to e.g. 0.94 ETH, the vault would lose ~6.1% of collateral value per ETH swapped – beyond the typical 1-2% `premium`, though inside the 10% `MAX_PREMIUM` ceiling. No on-chain divergence detection exists; mitigation is the reactive `pause()`.

Recommendation

Cross-check the Aave wstETH oracle against `stEthPerToken` inside the swap and revert on divergence above a configurable threshold (e.g. 0.5%); absent that, monitor Aave governance for wstETH oracle changes and preemptively `pause()` on proposals that alter the oracle shape.

Client's Commentary:

The current Aave wstETH oracle tracks `stEthPerToken` by design; divergence would require a governance change to the oracle. Such a change is observable and would be mitigated by a reactive `pause()` before it takes effect.

L-4	<code>availableWstETHLiquidity</code> is an upper bound; <code>maxSwapToWstETH</code> result can shrink between quote and tx		
Severity	Low	Status	Acknowledged

Description

`availableWstETHLiquidity` returns `min(balance, allowance)` `ATokenWstETHSwap.sol:166-170`; each swap decreases the allowance. A UI that submits `amountIn` mapping to `fullAmount` $\approx$ `availableWstETHLiquidity` can race another user's swap and revert with `InsufficientLiquidity`. The NatSpec directs integrators to `maxSwapToWstETH` for the debt-ceiling picture but does not document the race condition.

Recommendation

Extend the existing `maxSwapFromDebtCeiling` NatSpec caveat `ATokenWstETHSwap.sol:174-177` to `availableWstETHLiquidity` and `maxSwapToWstETH`, flagging them as upper bounds that can shrink between quote and tx.

Client's Commentary:

This may become an issue only when the user's intended swap size approaches the current `availableWstETHLiquidity` – if there's meaningful headroom, concurrent swaps rarely consume enough to cause a revert.

3. About MixBytes

MixBytes is a leading provider of smart contract auditing and blockchain security research, helping Web3 projects enhance their security and reliability.

Since its inception, MixBytes has been dedicated to safeguarding innovation in **DeFi** through rigorous audits and cutting-edge technical research.

Our team brings together experienced engineers, security auditors, and blockchain researchers with deep expertise in smart contract security and protocol design.

With years of proven experience in Web3, MixBytes combines in-depth technical excellence with a proactive, security-first approach.

Why MixBytes

- **Proven Track Record:** Trusted by leading blockchain protocols such as **Lido**, **Aave**, **Curve**, **1inch**, **Fluid**, **Gearbox**, **Resolv**, and others. MixBytes has successfully secured billions of dollars in digital assets.
- **Technical Expertise:** Our auditors and researchers hold advanced degrees in cryptography, cybersecurity, and distributed systems, backed by years of hands-on experience.
- **Innovative Research:** MixBytes actively contributes to blockchain security research and open-source tools, sharing insights with the global community.

Our Services

- **Smart Contract Audits:** Comprehensive security assessments of DeFi protocols to detect and mitigate vulnerabilities before deployment.
- **Blockchain Research:** In-depth technical studies, economic and protocol-level modeling for Web3 projects.
- **Custom Security Solutions:** Tailored frameworks and advisory for complex decentralized systems and blockchain ecosystems.

Contact Information



<https://mixbytes.io/>



https://github.com/mixbytes/audits_public



<https://x.com/MixBytes>



hello@mixbytes.io